



CHECKPOINT 5 · PROTÓTIPO FUNCIONAL

PROTÓTIPO FUNCIONAL

FIAP · Mobile Development & IoT · Engenharia de Software

Arquitetura, decisões técnicas, dados mockados, ambiente de teste e banco de dados do protótipo do **Claquete**.

Disciplina: Mobile Development & IoT · FIAP · 3º ano de Engenharia de Software

Etapas: Checkpoint 5 — Protótipo funcional

1. O QUE EXISTE AGORA

O protótipo é navegável de ponta a ponta. É possível percorrer uma rodada inteira: receber a vez, escolher o filme, confirmar presença, dar a nota, ver o veredito e encontrar a rodada refletida no placar da temporada.

ENTREGUE	ONDE ESTÁ
8 telas navegáveis	<code>app/</code>
Regras de negócio isoladas e testadas	<code>src/domain/</code>
Dados mockados em JSON local	<code>src/data/mock/</code>
39 testes automatizados	<code>src/domain/__tests__/</code> e <code>src/data/__tests__/</code>
Banco de dados em Postgres	<code>supabase/schema.sql</code>
Evidências de execução	<code>docs/evidencias/</code>

2. ARQUITETURA

O projeto está separado em quatro camadas, e a separação não é decorativa: é o que permite que o mesmo aplicativo rode com JSON local ou com Postgres sem que nenhuma tela saiba a diferença.

<code>app/</code>	telas e navegação (Expo Router)
└─ <code>usa</code>	
<code>src/store/</code>	estado da aplicação (Zustand)
└─ <code>usa</code>	
<code>src/data/</code>	contrato de repositório + duas implementações
└─ <code>usa</code>	
<code>src/domain/</code>	regras de negócio puras, sem React e sem rede

2.1 Domínio — as regras que definem o produto

Funções puras, sem dependência de React, de rede ou de banco. É onde moram as três decisões descritas no [documento de escopo](#):

ARQUIVO	RESPONSABILIDADE
<code>rotation.ts</code>	De quem é a vez, qual a rodada em jogo e se o prazo do curador venceu
<code>voting.ts</code>	Quem votou, quantos faltam, quando as notas são reveladas e qual a média
<code>standings.ts</code>	O placar da temporada, com empates dividindo posição

Ser função pura é o que torna essas regras testáveis sem emulador, sem banco e sem interface — e é por isso que os testes rodam em menos de meio segundo.

2.2 Dados — um contrato, duas implementações

`ClubRepository` declara tudo o que o aplicativo precisa da camada de dados. Duas implementações satisfazem o mesmo contrato:

- `mockRepository` — JSON local, em memória
- `supabaseRepository` — Postgres, via Supabase

Quem decide qual usar é `getRepository()`: havendo credenciais no `.env`, o Supabase assume; não havendo, o JSON local responde.

Por que o retorno ao mock é proposital. O protótipo precisa rodar no emulador, no navegador e em uma máquina que nunca viu as credenciais. Um `.env` ausente não pode ser o motivo de uma demonstração falhar.

2.3 Estado

`useClubStore`, em Zustand. Toda alteração passa pelo repositório e recarrega o clube em seguida, então a tela sempre mostra o que o armazenamento de fato tem — não existe estado otimista para divergir do banco.

3. DECISÕES TÉCNICAS

DECISÃO	ALTERNATIVA CONSIDERADA	POR QUE ASSIM
Expo Router para navegação	React Navigation puro	Rotas por arquivo, com tipagem automática. O caminho da tela é o caminho do arquivo, o que elimina uma tabela de rotas para manter
Zustand para estado	Context API, Redux	O estado é pequeno e quase todo derivado. Context re-renderiza demais e Redux traria cerimônia sem contrapartida neste tamanho
Regras puras isoladas	Lógica dentro dos componentes	Permite testar o produto sem montar tela. As três decisões que sustentam o app são exatamente as três que têm teste

DECISÃO	ALTERNATIVA CONSIDERADA	POR QUE ASSIM
Contrato de repositório	Chamar o Supabase direto das telas	Trocar a origem dos dados vira uma linha. Sem isso, a integração do banco tocaria em todas as telas
react-native-svg para ícones	Biblioteca de ícones pronta	Os ícones são os do CP4, inclusive a marca na aba do clube. Um conjunto pronto traria outro traço
Datas escritas à mão	<code>Intl.DateTimeFormat</code>	As abreviações do Intl variam entre versões do Hermes, e essas datas aparecem em toda tela — precisam ser idênticas no emulador, no navegador e no aparelho
Jest em ambiente Node	Preset <code>jest-expo</code>	O preset carrega módulos nativos que não existem em Node e falha. Como os testes cobrem regras puras, o runtime nativo é dispensável
Pôsteres locais	Buscar do TMDb agora	O CP5 pede dados mockados. A troca para a API pública é o bônus previsto para o CP6, e a camada de serviços já está no lugar

4. DADOS MOCKADOS

O clube de demonstração está em `club.json` e o catálogo em `movies.json`.

A temporada não começa vazia: quatro rodadas já encerradas dão história ao clube — placar preenchido, estante com filmes e resenhas — e a quinta rodada começa **na vez do usuário**, que é o ponto onde a demonstração pega o fluxo.

RODADA	CURADOR	FILME	MÉDIA
1	Marina	Ainda Estou Aqui	9.2
2	Bia	Central do Brasil	8.4
3	João	Tropa de Elite	7.6
4	Gabriel	Cidade de Deus	8.2
5	Felipe	<i>aguardando escolha</i>	—

As médias não estão escritas em lugar nenhum. Elas são calculadas pelo domínio a partir dos votos individuais — apagar um voto muda a média e muda o placar, como mudaria num aplicativo de verdade.

4.1 O que é simulado, e por quê

O protótipo roda com um usuário só. Sem alguma simulação, a regra central do produto — *as notas só aparecem quando todos votam* — nunca poderia ser exercida: a rodada ficaria parada esperando quatro pessoas que não existem.

Então duas coisas acontecem por conta do protótipo, e ambas estão marcadas no código como `Prototype only`:

- ao escolher o filme, **parte do clube confirma presença**
- ao abrir a votação, **os outros membros votam**, com notas fixas

As notas simuladas são fixas de propósito: assim toda execução da demonstração chega ao mesmo veredito, e quem apresenta sabe o que vai aparecer na tela.

Elas vivem em um módulo só, `src/data/simulation.ts`, importado pelos dois repositórios. Enquanto cada um tinha a sua cópia, a mesma rodada fechava em médias diferentes dependendo de onde os dados vinham — o protótipo local batia com o CP4 e o banco não. Um teste agora prende esse número.

4.2 FIDELIDADE ÀS TELAS DO CP4

Os dados mockados foram escritos para que o protótipo reproduza as telas conceituais do [documento de telas](#). O que bate exatamente:

- **O filme da rodada** — Cidade de Deus, com o mesmo pôster, ano, gêneros e duração
- **O curador** — Gabriel, e a rodada 5 de 8
- **A estante** — Ainda Estou Aqui na rodada 4 e Tropa de Elite na rodada 3
- **As confirmações** — 3 de 5, com o botão de confirmar presença ainda disponível
- **Os votos e as resenhas do veredito** — os cinco, palavra por palavra
- **O catálogo do curador** — os mesmos filmes e as mesmas plataformas do CP4
- **A estrutura do placar** — pódio, quarto e quinto lugar, e o "ainda não foi curador"

A comparação, tela a tela

As telas do protótipo foram fotografadas em execução e postas lado a lado com as conceituais. A conferência achou diferenças que não eram de dados, e elas foram corrigidas:

TELA	O QUE ESTAVA DIFERENTE
Abertura	A frase de efeito quebrava no meio (" <i>Toda semana um escolhe. Todo / mundo julga.</i> ")
Vez do curador	O aviso de prazo dizia " <i>Escolha faltam 7 dias ou a vez passa</i> ", sem relógio; a busca não tinha lupa; a sessão saía sem o dia da semana
Sua nota	As notas usavam a fonte de texto, centralizadas, e o aviso tinha borda âmbar em vez de cinza
Veredito	A contagem de votos ficava na coluna da média, sem o avatar do curador; a lista vinha sem divisórias e fora de ordem; havia dois botões onde o desenho tem um
Clube	A tira do topo mostrava três avatares, e o desenho conta o resto num "+3"

O que continua diferente é de **dado**, não de desenho: as datas e a contagem regressiva seguem o calendário da temporada, e a busca do curador não repete os títulos do CP4 porque o clube já assistiu a eles — a tela esconde o que já foi visto, que é justamente o comportamento correto.

Duas contradições no material do CP4

As telas conceituais foram desenhadas uma a uma e, postas lado a lado, não fecham entre si. O protótipo precisou escolher, porque **um banco de dados não aceita duas verdades**:

CONTRADIÇÃO	O QUE O CP4 MOSTRA	O QUE O PROTÓTIPO FAZ
Posição da Marina no rodízio	A estante diz "rodada 4 · escolha da Marina" e, três centímetros acima, "Rodada 6 é da Marina" — com cinco membros, ela não ocupa a 4ª e a 1ª posição ao mesmo tempo	Mantém a estante (dois filmes com nota, mais visíveis) e a rodada 6 fica com a Bia
Quem nunca curou	O placar mostra o João como "ainda não foi curador" depois de cinco rodadas — com cinco membros, cinco rodadas cobrem todo mundo	Quem ainda não curou é o Gabriel, que é justamente quem está com a rodada em jogo

Há ainda um detalhe aritmético: a nota **7.1** do placar do CP4 não sai de cinco notas inteiras — precisaria somar 35,5. No protótipo a média é **calculada** a partir dos votos, não digitada, então números impossíveis simplesmente não aparecem. É uma diferença que só existe porque agora há uma conta de verdade por trás do número.

Ver o aplicativo como outro membro

A tela de **Perfil** permite trocar de usuário. Não é enfeite: o Claquete é um produto de grupo apresentado em um aparelho só, e telas que existem apenas para o curador — como a escolha do filme — ficariam inalcançáveis sem isso.

5. AMBIENTE DE TESTE

```
npm test
```

São **39 testes em 4 conjuntos**, cobrindo as regras e a coerência dos dados:

CONJUNTO	O QUE GARANTE
<code>rotation.test.ts</code>	O rodízio segue a ordem, dá a volta corretamente, recusa entrada inválida e o prazo do curador vence na hora certa
<code>voting.test.ts</code>	As notas ficam fechadas até o último voto, a média arredonda para uma casa e nota zero não se confunde com ausência de nota
<code>standings.test.ts</code>	O ponto vai para o curador e não para quem votou, empates dividem posição e quem nunca curou fica por último sem nota
<code>mock.test.ts</code>	Os dados mockados são coerentes: curador bate com o rodízio, voto só de quem é do clube, filme existe no catálogo, e os votos simulados fecham a rodada na média desenhada no CP4

O último conjunto merece explicação: ele testa **dados**, não código. Serve para que uma edição descuidada no JSON — um voto de quem não é do clube, um curador fora do rodízio — seja reprovada antes da apresentação, e não durante.

Este conjunto já pegou um erro real: uma semente temporária usada para fotografar a tela de votação ficou aplicada por engano, e o teste acusou.

5.1 Evidências

```
npm run web           # em um terminal, deixe rodando
node scripts/capture-evidence.mjs  # em outro
```

O script não monta mockup nenhum: ele abre o aplicativo servido pelo Expo num Chrome de 390×844, **toca nos botões na mesma ordem da apresentação** — escolher o filme, confirmar presença, votar, fechar a rodada — e fotografa cada tela depois que ela para de mudar. As imagens em `docs/evidencias/` são, portanto, o produto rodando com os dados do banco, e não uma reprodução.

O roteiro reinicia a demonstração no começo e no fim, então rodar o script devolve o banco exatamente onde o encontrou.

6. BANCO DE DADOS

O banco é **Supabase** (Postgres gerenciado). O modelo tem cinco tabelas:

TABELA	GUARDA
<code>clubs</code>	O clube, a temporada e a ordem do rodízio
<code>members</code>	Quem participa
<code>rounds</code>	Cada rodada: curador, filme, data da sessão, prazo e situação
<code>presences</code>	Quem confirmou presença em cada sessão
<code>votes</code>	Nota e resenha, com chave primária que impede voto duplicado

Duas escolhas de modelagem que valem registro:

- **A chave primária de `votes` é `(club_id, round_number, member_id)`**. Uma pessoa não vota duas vezes na mesma rodada — a regra é do banco, não do aplicativo. Votar de novo substitui a nota, via `upsert`.
- **O catálogo de filmes não está no banco.** A partir do CP6 ele vem da API do TMDB; guardá-lo agora seria criar uma tabela para descartar depois.

O **RLS** já fica **ligado** desde agora, com políticas permissivas para a chave anônima. Sem login, é o que o protótipo permite; no CP6, com autenticação, as políticas passam a verificar se quem escreve é membro do clube — e a estrutura já está pronta para isso.

6.1 Como configurar

1. Crie um projeto no [dashboard do Supabase](#)
2. Crie o esquema, de uma das duas formas:
 - **Pela CLI** (é como o projeto é mantido — veja 6.2)
 - **Pelo dashboard**: abra o **SQL Editor** e rode `supabase/schema.sql` inteiro
3. Vá em **Project Settings** → **API** e copie dois valores:
 - a **Project URL**
 - a chave `anon` / `public` (nas contas novas aparece como *publishable*)
4. Copie `.env.example` para `.env` e preencha:

```
cp .env.example .env
```

```
EXP0_PUBLIC_SUPABASE_URL=https://SEU-PROJETO.supabase.co  
EXP0_PUBLIC_SUPABASE_ANON_KEY=sua-chave-anon
```

5. Reinicie o servidor (`npm start`). A tela de **Perfil** mostra qual fonte de dados está ativa.

⚠ A chave **service_role** / **secret** nunca entra no aplicativo. Ela ignora todas as regras de segurança do banco; num repositório público, entregaria o banco a qualquer um. A chave **anon** é feita para ficar no cliente — quem controla o acesso são as políticas de RLS.

O `.env` está no `.gitignore` e não vai para o repositório.

6.2 Migrações pela CLI

O esquema é versionado em `supabase/migrations/` e o projeto está inicializado para a CLI (`supabase/config.toml`). Alterações no banco passam por ali, e não por SQL digitado no dashboard: assim o que está no repositório é o que está no banco, e a próxima pessoa consegue recriar o ambiente inteiro do zero.

Dois comandos são interativos e precisam ser rodados por quem tem a conta — o primeiro abre o navegador, o segundo pede a senha do banco:

```
npx supabase login  
npx supabase link --project-ref SEU-PROJECT-REF
```

Feito isso, aplicar as migrações é um comando:

```
npx supabase db push
```

O `project-ref` é a parte do meio da Project URL: `https://SEU-PROJECT-REF.supabase.co`.

7. COMO DEMONSTRAR

Com o aplicativo aberto na aba **Clube**, a rodada 5 está esperando a escolha:

1. **Escolher o filme** — a busca não mostra o que o clube já assistiu
2. **Bater a claquete** — a rodada passa a ter filme e data
3. **Confirmar presença** — parte do clube já confirmou
4. **Já assistimos** — abre a votação; os outros membros votam
5. **Dar minha nota** — escolha a nota e envie
6. O **veredito** aparece assim que o último voto entra
7. O **placar** mostra a rodada somada ao curador

A aba **Perfil** tem *Reiniciar demonstração*, que devolve o clube ao estado inicial — útil para apresentar duas vezes seguidas. O reset é do repositório, não da tela: com o banco ligado ele apaga e recria as rodadas, presenças e votos no Postgres a partir da mesma semente.

Se alguma escrita falhar — a rede caiu, a chave venceu —, a tela mostra o motivo. Vale dizer por que isso está aqui: enquanto os dados eram um JSON em memória, nada podia dar errado; com o banco do outro lado, uma falha silenciosa é pior do que uma falha, porque a tela recarrega igual e quem está apresentando acha que salvou.